Lab Report: Week 3 - Simple Pendulum

Chhaya Ramnani (UID: u8336188)

Lab partners: Tsz Wing Candice Ng, Chinatsu Komada

2026-08-12

Imports and Functions

All calculations were done in Python. Starting with imports:

```
import matplotlib.pyplot as plt
from uncertainties import ufloat
import numpy as np
from scipy import odr
import pandas as pd
```

Defining the gravity constant in Canberra:

```
g_accepted = 9.7976
```

To handle our data properly, we used a custom Python function (`odr_fit`) leveraging `scipy.odr`. Because our measurements have uncertainties in both the x and y axes, standard linear regression won't cut it. This function extracts the nominal values and errors from our `uncertainties` objects, sets up a linear model ($y = mx + c$), and performs an ODR to find the best-fit slope, its uncertainty, and the y-intercept.

```
def odr_fit(x_data, y_data, zero_intercept: bool = False):
    x = np.array([v.n for v in x_data])
    sx = np.array([v.s for v in x_data])
    y = np.array([v.n for v in y_data])
    sy = np.array([v.s for v in y_data])

    data = odr.RealData(x, y, sx=sx, sy=sy)

    if zero_intercept:
        linear_model = odr.Model(lambda p, x: p[0] * x)
        beta0 = [10.0]
    else:
```

```
        linear_model = odr.Model(lambda p, x: p[0] * x + p[1])
        beta0 = [10.0, 0.0]

    odr_result = odr.ODR(data, linear_model, beta0=beta0).run()

    slope = odr_result.beta[0]
    slope_err = odr_result.sd_beta[0]

    if zero_intercept:
        intercept = 0.0
        intercept_err = 0.0
    else:
        intercept = odr_result.beta[1]
        intercept_err = odr_result.sd_beta[1]

    return slope, slope_err, intercept, intercept_err, (x, y, sx, sy)
```

To visualize our results, we wrote a plotting function (`plot`) using `matplotlib`. This function creates a clear scatter plot of our data complete with error bars on both axes, overlays the best-fit line calculated from our ODR regression, and formats the legend to display our final spring constant value and its uncertainty.

```
def plot(x, y, sx, sy, slope, slope_err, intercept, xlabel, ylabel, title,
filename,  zero_intercept=False):
    x_fit = np.linspace(min(x), max(x), 100)
    y_fit = slope * x_fit + intercept

    plt.figure(figsize=(7, 5))
    plt.errorbar(
        x, y,
        xerr=sx, yerr=sy,
        fmt='o',
        color="navy",
        ecolor="crimson",
        capsize=3,
        label="Experimental Data"
    )
    plt.plot(
        x_fit, y_fit,
        "k--",
        label=f"Fit: Slope = {slope:.3f} ± {slope_err:.3f}"
    )
    plt.xlabel(xlabel)
```

```
    plt.ylabel(ylabel)
    plt.title(title)
    plt.grid(True, alpha=0.6)
    plt.legend()
    plt.tight_layout()
    plt.savefig(filename, dpi=300)
    plt.show()
```

To check if both methods gave consistent results, we run a sigma test (`sigma_test`). It calculates the statistical difference between the two spring constant values relative to their combined uncertainties. If the difference is less than or equal to 3 standard deviations (3σ), it indicates our results agree within experimental error.

```
def sigma_test(k1, sk1, k2, sk2):
    sigma_diff = abs(k1 - k2) / np.sqrt(sk1**2 + sk2**2)
    print("SIGMA TEST")
    print(f"Static k  : {k1:.3f} +/- {sk1:.3f} N/m")
    print(f"Dynamic k : {k2:.3f} +/- {sk2:.3f} N/m")
    print(f"Discrepancy: {sigma_diff:.2f} sigma")

    if sigma_diff <= 3.0:
        print("results AGREE within experimental uncertainty")
    else:
        print("results DISAGREE")
```

Pre-lab Questions

Question 1-

Using equation (3),

$$T = 2\pi\sqrt{\frac{L}{g}}$$

$$\frac{T}{2\pi} = \sqrt{\frac{L}{g}}$$

$$\left(\frac{T}{2\pi}\right)^2 = \frac{L}{g}$$

$$g = \frac{4(\pi)^2 L}{T}$$

Question 2-

Using the above obtained equation for g, and substituting measured values with their absolute errors, $L \pm \sigma_L$ and $T \pm \sigma_T$,

$$g \pm \sigma_g = \frac{4(\pi)^2(L \pm \sigma_L)}{(T \pm \sigma_T)}$$

Rearranging,

$$\sigma_g = \pm \left(\frac{4(\pi)^2(L \pm \sigma_L)}{(T \pm \sigma_T)} - g \right)$$

Experiment 1

Aim

The aim of this experiment is to measure the period of a simple pendulum with respect to some (small, medium and large) initial release angle to highlight the connection between angle, oscillatory period, and acceleration due to gravity.

Method

For small angles, the period of a simple pendulum is given by:

$$T = 2\pi\sqrt{\frac{L}{g}},$$

where L is the length of the string atttached to a point of mass m.

For larger angles up to approximately 40°, analytical corrections must be applied:

$$T = 2\pi\sqrt{\frac{L}{g}}\left(1 + \frac{1}{16}\theta_0^2\right)$$

where θ_0 is the initial angle in radians, when the pendulum is released from rest.

Rearranging this expression isolates the gravitational acceleration $ g:$g = \frac{4\pi^2 L}{T^2}\left(1 + \frac{1}{16}\theta_0^2\right)^2$

Procedure

We first set up the g-clamp and fix it onto the table along with a rigid pendulum clamp. Attaching the string to the metal bob and dangling it from the pendulum clamp, we measure the length of the string.

We then drag our pendulum to an initial angle, and then measure the time it takes for the pendulum to complete an oscillation 10 times. Once done, we take our measurements and divide them by 10 to get the period for each swing.

As an effort to minimize human error, we had the person that swings the pendulum be the person to start the timer, as they can let go of the bob as soon as they start the timer.

We repeat this process for the 3 angles we're asked to measure: 5, 25, and 40, with 3 trials each.

Results

As previously stated, all calculations and interpretations were done using python. I've defined all my used fuctions in the **Imports and Functions** section. Lets starttt

Our measured length of the string turned out to be $51.2cm$ or $0.512m$. We measured the length of the string in increments of $0.5cm$, so we account the uncertainty as half of that, $0.25cm$. Uh there are 3 values because i'm using arrays instead of ufloats so its a bit messy.

```
L_p1_cm = [51.2, 51.2, 51.2]  # measured length
sigma_L = 0.25 / 100  # m
```

For each angle, we dangled the pendulum for, we took 3 runs of 10 oscillations. each array has 3 values, which is the time taken by the pendulum to swing 10 times, taken thrice.

Considering an uncertainty of 0.20 seconds. It is safe to assume so as our measured values lie within this range of each other, and it also nears the average human reaction time.

As for the angles, since we were measuring by eye, a 0.5 degree error is quite prone to occur. Hence accounted for as well.

```
angles_deg = [5.0, 25.0, 40.0]
time_10_oscillations_trials = [
    [14.53, 14.41, 14.36],
    [14.67, 14.66, 14.68],
    [14.88, 14.92, 14.95]
]
sigma_t = 0.20 # seconds
sigma_theta_deg = 0.5 #degrees
```

The following code block does the following:

- it grabs the theta values and convert them into ufloats with their uncertainties. After doing so, it performs a unit conversion to radians.
- the mean is taken of the 3 time measurement trials of each degree as well as their uncertainty is accounted for. `total_time_err` is our combined per-trial timing uncertainty- it's picking the larger of two uncertainty estimates rather than combining both, which is good as it reports the limiting factor.
- then the time measurements and their uncertainties > ufloats > converted into Time periods by dividing by 10.

- as for the correction factor stuff — for angles below 10 deg, we don't bother applying it, but for 25 deg and 40 deg, we multiply through by $\left(1 + \frac{1}{16}\theta_0^2\right)$ Since we're now treating θ_0 as a ufloat too, the correction factor carries its own tiny uncertainty that propagates through automatically.

(please ignore the very annoying warning blocks i'm trying to find a solution to them)

```
g_results_p1 = []
table_data = []

print("~~~PART 1 RESULTS~~~")
for i in range(len(angles_deg)):
    theta_deg = ufloat(angles_deg[i], sigma_theta_deg)
    theta_rad = theta_deg * np.pi / 180.0

    trials = time_10_oscillations_trials[i]
    mean_time = np.mean(trials)
    std_err_time = np.std(trials, ddof=1) / np.sqrt(len(trials))
    total_time_err = max(std_err_time, sigma_t)

    L = ufloat(L_p1_cm[i] / 100.0, sigma_L)
    t_total = ufloat(mean_time, total_time_err)
    T = t_total / 10.0  # period for one oscillation

    # angle correction factor
    if theta_deg.n >= 10.0:
        correction_factor = 1.0 + (1.0 / 16.0) * (theta_rad**2)
    else:
        correction_factor = ufloat(1.0, 0.0)

    g_val = 4.0 * (np.pi**2) * L * (correction_factor**2) / (T**2)
    g_results_p1.append(g_val)

    table_data.append({
        "Angle (°)": f"{theta_deg.n:.1f} ± {theta_deg.s:.1f}",
                        "Correction   Factor":   f"{correction_factor.n:.4f}   ±
{correction_factor.s:.4f}",
        "g (m/s²)": f"{g_val.n:.3f} ± {g_val.s:.3f}",
        "Discrepancy (σ)": f"{abs(g_val.n - g_accepted) / g_val.s:.2f}"
    })

df_p1 = pd.DataFrame(table_data)
df_p1
```

```
~~~PART 1 RESULTS~~~
```

```
/Users/chhaya/Documents/uniwork-cr/.venv/lib/python3.9/site-packages/
uncertainties/core.py:1024: UserWarning: Using UFloat objects with std_dev==0
may give unexpected results.
  warn("Using UFloat objects with std_dev==0 may give unexpected results.")
```

	Angle (°)	Correction Factor	g (m/s²)	Discrepancy (σ)
0	5.0 ± 0.5	1.0000 ± 0.0000	9.703 ± 0.273	0.35
1	25.0 ± 0.5	1.0119 ± 0.0005	9.617 ± 0.267	0.68
2	40.0 ± 0.5	1.0305 ± 0.0008	9.646 ± 0.263	0.58

All three agree with $g = 9.7976$ well within 3σ. So the correction is doing its job — even at 40 deg, we're not seeing the pendulum "break." Worth noting though, the correction factor's own uncertainty is basically negligible next to our timing uncertainty (0.0005–0.0008 vs the ~0.02s timing error dominating everything) — so whether we bother propagating σ_θ through at all barely changes our final g uncertainty. It was merely a for-the-process thing.

Experiment 2

Aim

To investigate how the period of a simple pendulum depends on its length (L) under small-angle conditions ($\theta_0 < 10^\circ$), and to obtain a highly precise value for g using a linearised graph.

Method

Starting from Equation 3,

$$T = 2\pi\sqrt{\frac{L}{g}}$$

$$\frac{T}{2\pi} = \sqrt{\frac{L}{g}}$$

Squaring both sides yields a linear relationship ($y = mx + c$):

$$T^2 = \left(\frac{4\pi^2}{g}\right)L$$

where

$$y = T^2(\mathrm{s}^2),$$

$$x = L(\mathrm{m}),$$

$$m(gradient) : \frac{4\pi^2}{g} \Longrightarrow g = \frac{4\pi^2}{\mathrm{slope}}$$

Procedure

Same setup as Part 1— g-clamp, rigid pendulum clamp, string with a metal bob. But this time we keep the release angle small, 10 deg and instead vary the string length across 4 values, $20, 40, 60 & 80 cm$ For each length we again time 10 oscillations, repeated 3 times, and divide by 10 to get the period.